# M-Pesa Fraud Detection System

A Real-Time Transaction Decisioning Engine

**Anthony Ng'ang'a**
**Data Scientist & ML Engineer**

# Kenya Runs on M-Pesa


- Launched 2007 — solved a systemic banking access gap
- Only requirement: a SIM card
- Agents in the deepest rural areas of Kenya
- Transactions as low as KSh 1
- From mama mboga to corporate payroll

# KSh 2.3 Billion Lost
## To Fraud Every Year

37.15 Billion transactions annually
KSh 38.29 Trillion total transaction value
KSh 6.3 Million stolen every single day
21,246 successful scam operations annually
Median loss per victim: KSh 108,132

# Fraudsters Are **Sophisticated and Adaptive**

## Common attack vectors:

**01 SIM Swap**
Hijack number, drain account

**02 Social Engineering**
Trick users into sending money

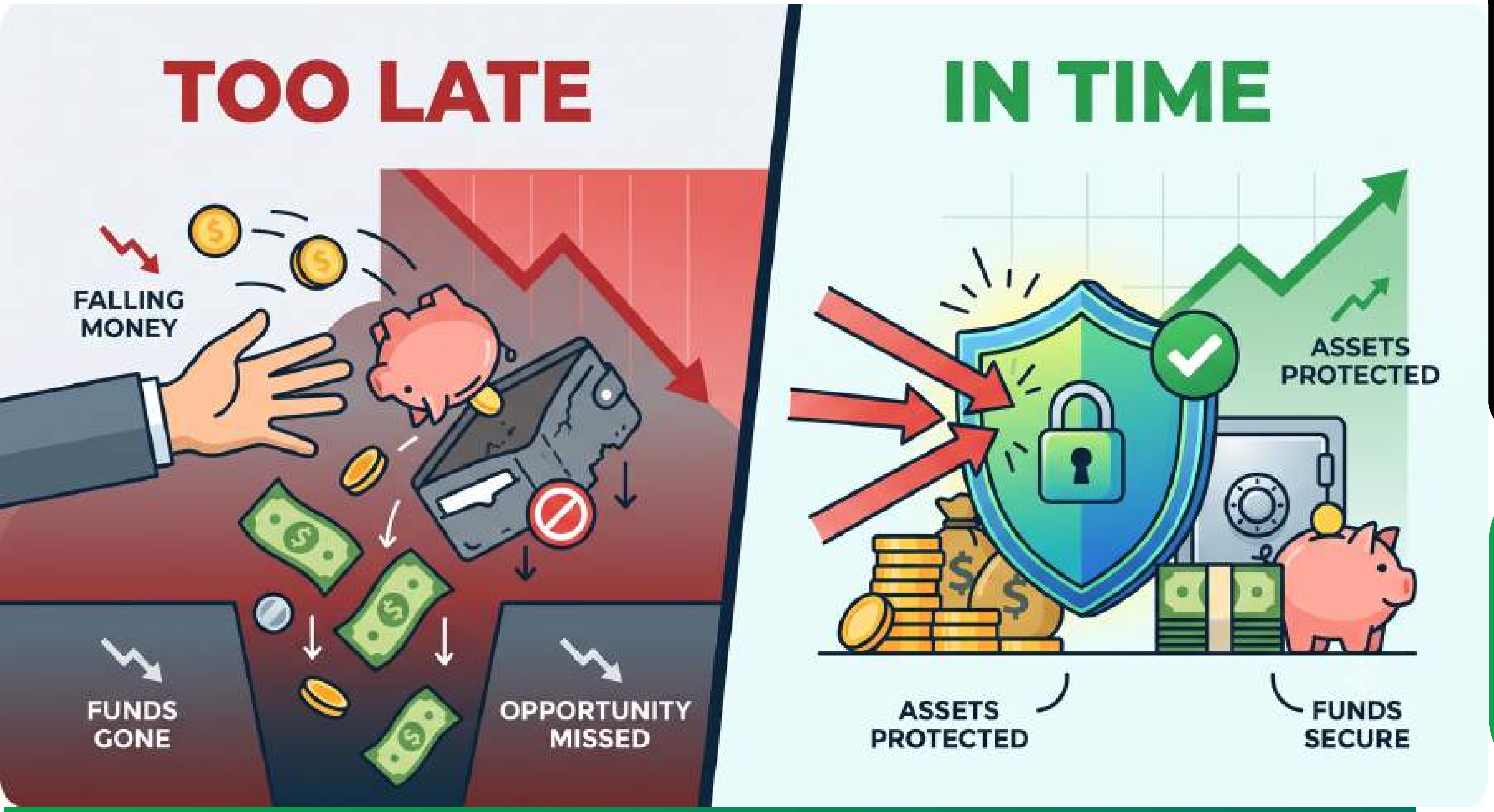
**03 Fraudulent Merchant Transactions**
Fake till/paybill numbers

**04 Number Masking**
Impersonate known contacts

**05 Fake Balance SMS**
Create false urgency

# Detecting Fraud After The Fact  Solves Nothing

**Traditional approach:**
Transaction completes → Fraud flagged → Investigation → Maybe recovery




The real cost:

- Money already gone
- Customer trust damaged
- Recovery rate near zero for M-Pesa P2P fraud

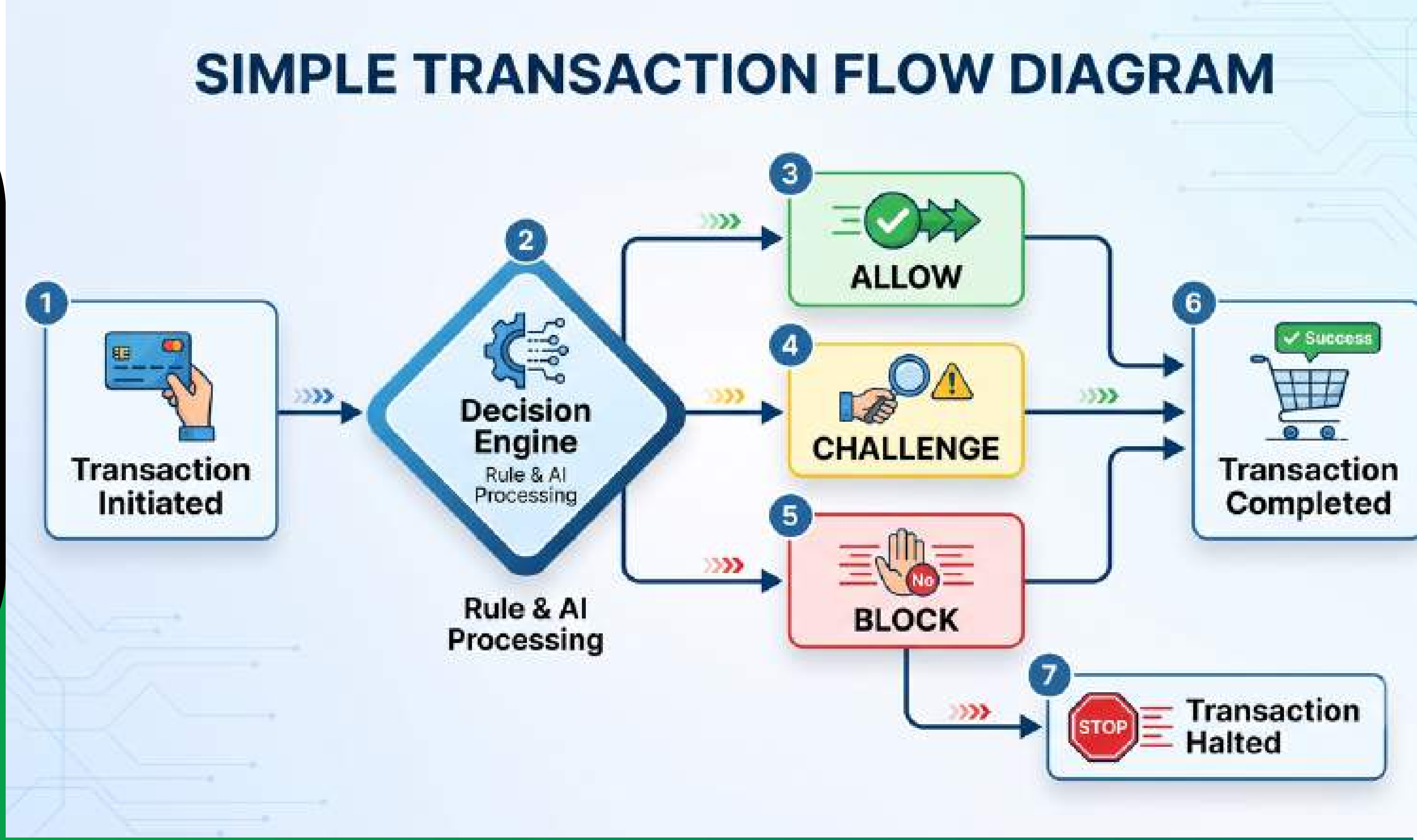
**What's needed:**
Intercept before settlement. Not after.

# A Real-Time Transaction
# Decisioning Engine

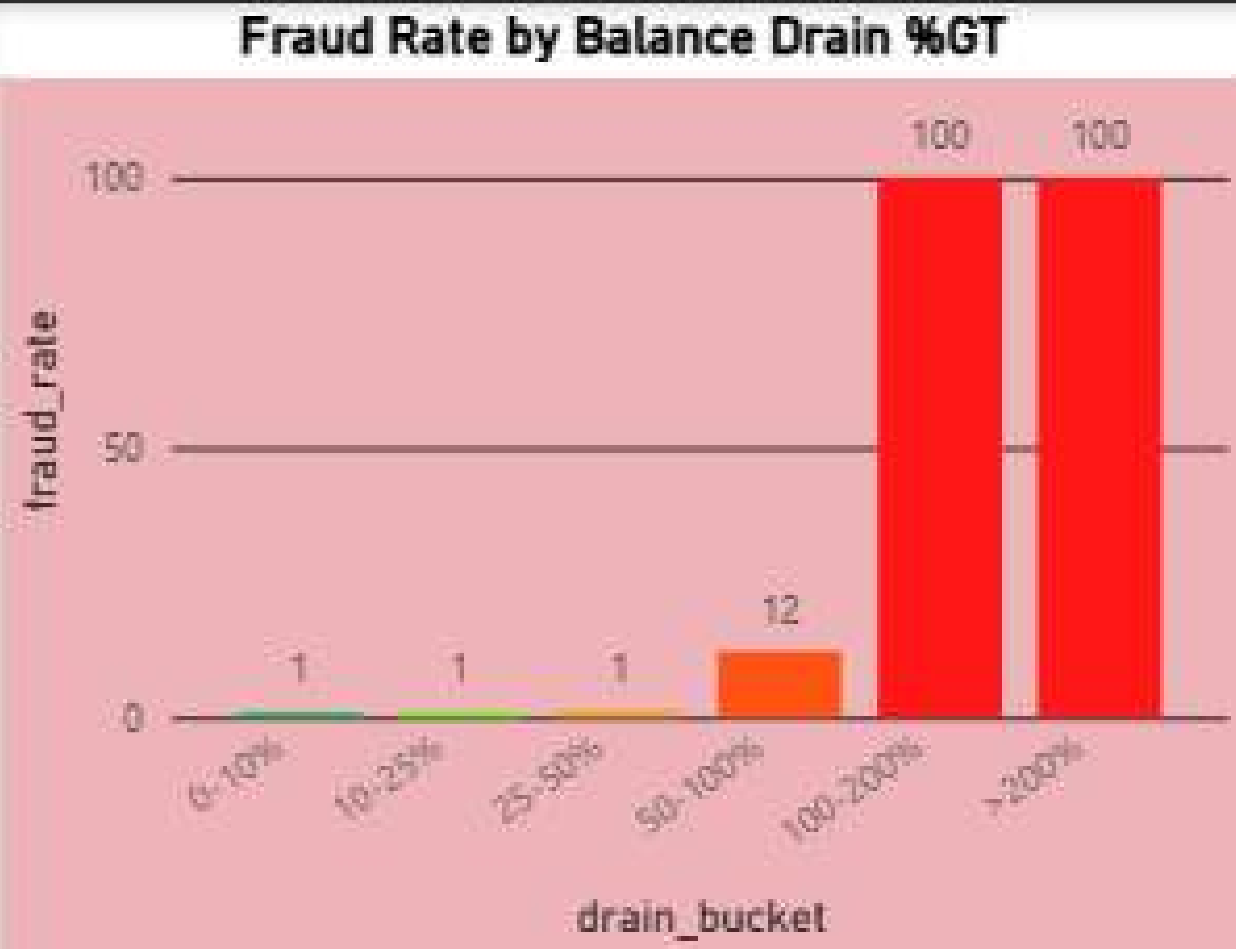
The model sits between transaction initiation and completion. Decision returned in milliseconds — before money moves.

M-Pesa processes 3,000–4,000 transactions per second. The decisioning layer must operate within a 10–15ms latency window to be viable at scale.

| | |
|---|---|
| ALLOW | Low risk — transaction proceeds |
| CHALLENGE | Elevated risk — OTP verification triggered |
| BLOCK | High risk — transaction halted before settlement |



# Two Signals Predict Fraud
# With Near-Certainty




Balance Drain % — how much of the sender's balance is the transaction?

- Below 50% drain → fraud rate ~1%
- Above 100% drain → fraud rate 100%

Account Emptied — does the transaction zero out the account?

- 2,071 transactions emptied accounts
- Every single one was fraudulent — no exceptions

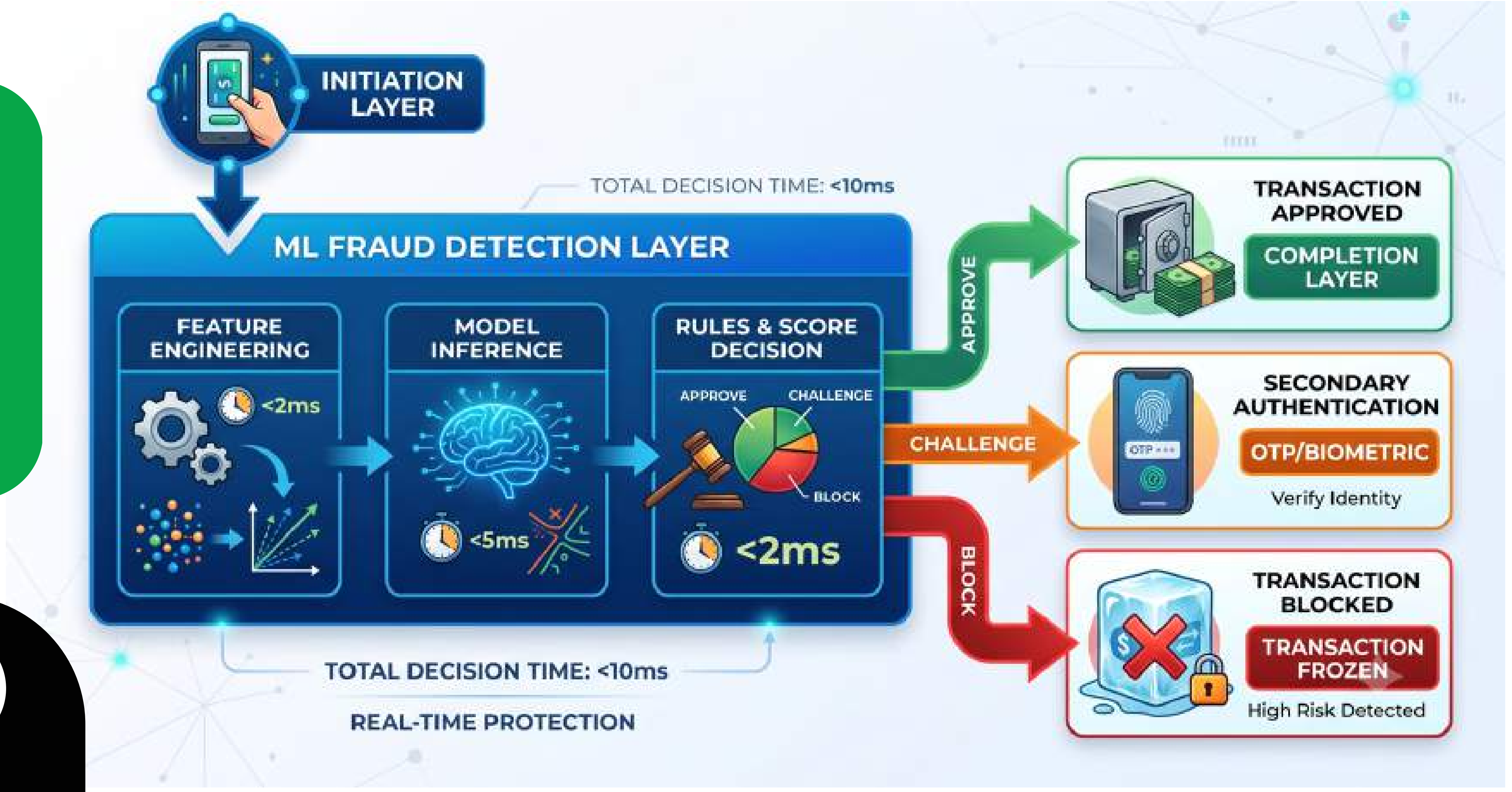
# How the Model Works

Initiation Layer

**ML Fraud Detection Layer**

→ Feature Engineering (<2ms)
→ Model Inference (<5ms)
→ Rules & Score Decision (<2ms)

| | |
|---|---|
| ALLOW → | Completion Layer |
| CHALLENGE → | OTP/Biometric |
| BLOCK → | Transaction Frozen |



# Results That Generalise to Unseen Data

| Metric | Score |
| --- | --- |
| Recall (fraud) | 0.68 |
| Precision (fraud) | 0.03 |
| Accuracy | 0.35 |

catches 68% of fraud

known limitation (synthetic data)

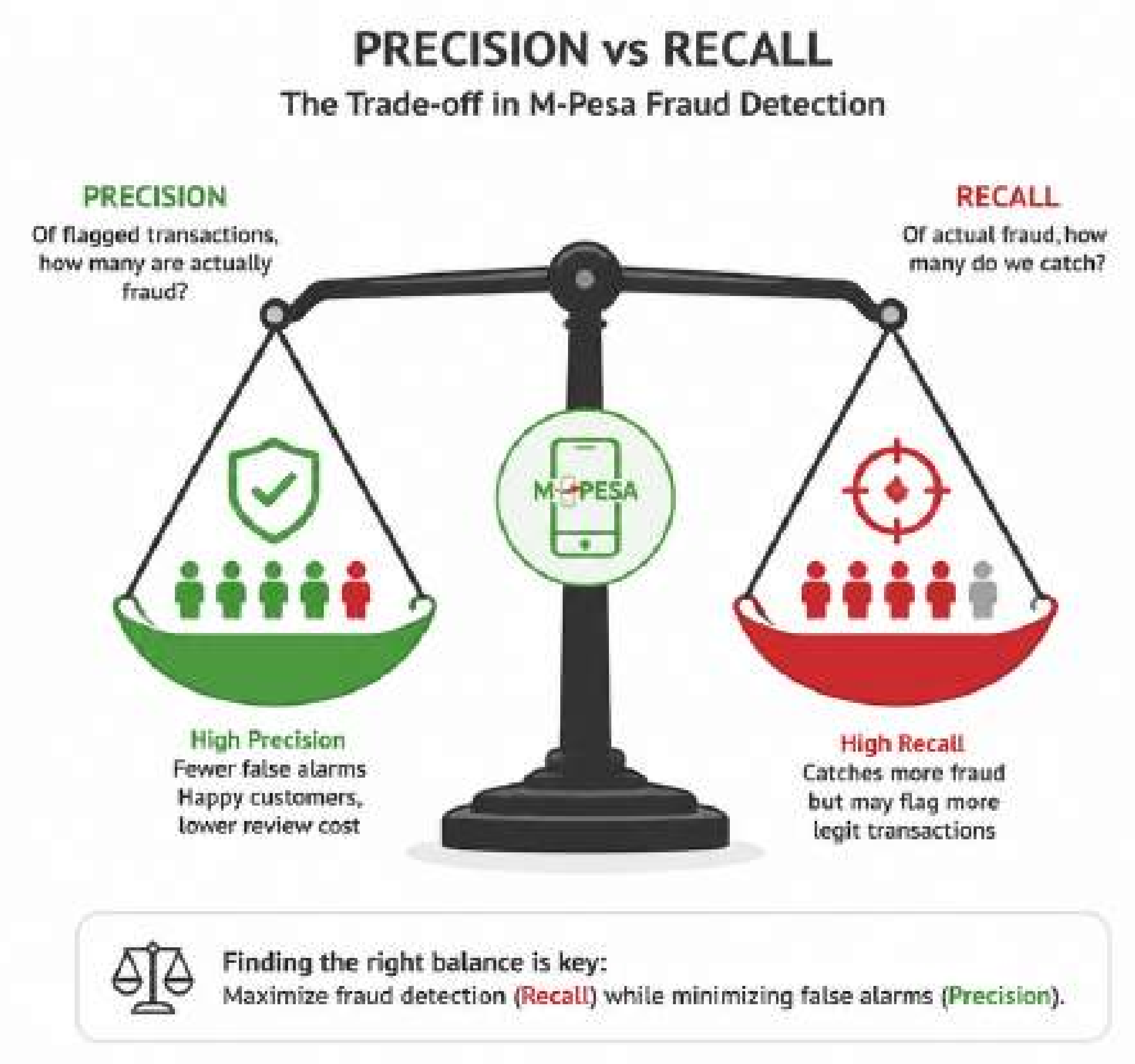
*Evaluated against 10,000 completely unseen transactions with original 97/3 class ratio preserved:*

**Key finding**

Performance on unseen data matched training performance exactly — the model generalises, it does not memorise

# What This Model Can and Cannot Do

| LIMITATION | REASON |
| --- | --- |
| **Precision stuck at 0.03** | Synthetic data — uniform fraud distribution |
| **High false positive rate** | Trade-off for maximising recall |
| **No SIM swap detection** | Not in dataset |
| **Results on real data may vary** | Synthetic-on-synthetic SMOTE effect |




**Why this matters:**

- An honest model card builds more trust than inflated metrics.
- Real-world deployment requires retraining on live transaction data.

# From Notebook to Production

**BACKEND**
Data → Feature Engineering → Model Training → FastAPI Endpoint → Render Deployment
**Live endpoints:**
- GET /health_status
- POST /predict
- mpesa-fraud-detection-system.onrender.com/

**FRONTEND**
Modernized web UI/UX build on Node.js, links and connects to the render webservice *mpesa-fraud-detection-system.onrender.com/* and deployed on vercel
- **LINK: *https://mpesa-fraud-detection-system.vercel.app/***

**Production scale would require:**

- Kafka for streaming ingestion
- Redis for in-memory feature store
- ONNX Runtime for sub-5ms inference
- Drools for dynamic rule engine

**TECH STACK:**

XGBOOST | FASTAPI | REACT | RENDER
SCIKIT-LEARN | NEXT.JS | VERCEL | POWER-BI

# What's Next: Built to Evolve

**Immediate next steps:**

- Retrain on real M-Pesa transaction data
- Add SIM swap flag and agent transaction features
- Extend to bank transaction channel

**Longer term:**

- Online learning for continuous model adaptation
- Frontend dashboard for transaction monitoring
- Multi-channel expansion (Airtel Money, bank transfers)

**The infrastructure is built.**

**The pipeline is live.**

**The model is ready for real data.**

# Thank You

anthonyngangachege.vercel.app